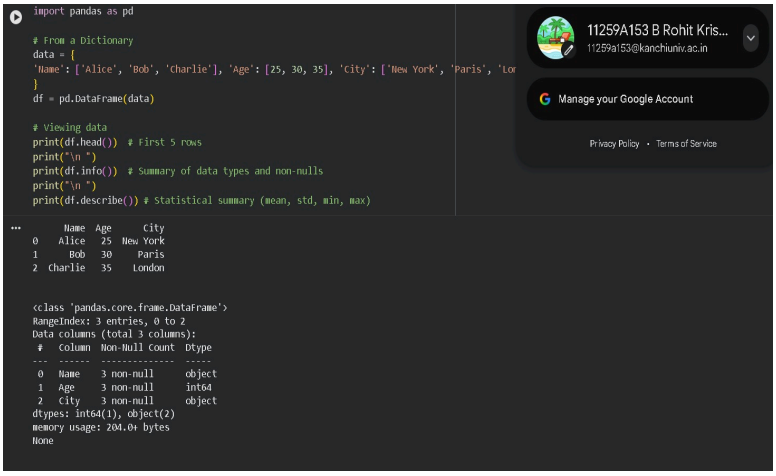
Pandas (Python Data Analysis Library)

A. Creating Data Frames

Aim: To create a Pandas DataFrame from a dictionary and display its contents, data types, and statistical summary.

Algorithm:

1. Import the Pandas library.
2. Create a dictionary containing Name, Age, and City.
3. Convert the dictionary into a DataFrame using pd.DataFrame().
4. Display the first five rows using df.head().
5. Display column information using df.info().
6. Display the statistical summary using df.describe().



```
import pandas as pd

# From a Dictionary
data = {
'Name': ['Alice', 'Bob', 'Charlie'], 'Age': [25, 30, 35], 'City': ['New York', 'Paris', 'Lor
}
df = pd.DataFrame(data)

# Viewing data
print(df.head())  # First 5 rows
print("\n ")
print(df.info())  # Summary of data types and non-nulls
print("\n ")
print(df.describe()) # Statistical summary (mean, std, min, max)
```

```
      Name  Age     City
0    Alice   25  New York
1      Bob   30     Paris
2  Charlie   35    London


<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3 entries, 0 to 2
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Name    3 non-null      object
 1   Age     3 non-null      int64
 2   City    3 non-null      object
dtypes: int64(1), object(2)
memory usage: 204.0+ bytes
None
```

11259A153 B Rohit Kris...
11259a153@kanchiuniv.ac.in
Manage your Google Account
Privacy Policy · Terms of Service

Result: Thus, a Pandas DataFrame was created successfully, and its contents, data types, and statistical summary were displayed.

B. Selecting and Filtering

Aim: To select columns, filter rows based on a condition, and access specific rows and values using iloc and loc in Pandas.

Algorithm:

1. Use the DataFrame created in Program 1.
2. Select the Age column using df['Age'].
3. Filter people whose age is greater than 25 using df[df['Age'] > 25].
4. Retrieve the first row using df.iloc[0].
5. Retrieve the name at index 0 using df.loc[0, 'Name'].
6. Display the selected column, filtered rows, first row, and specific value.




```
# Selecting a column
ages = df['Age']
print("Ages column:")
print(ages)

print("\n")

# Filtering rows
above_25 = df[df['Age'] > 25]
print("People with Age > 25:")
print(above_25)

print("\n")

# iloc (integer-based)
row_0 = df.iloc[0]
print("First row using iloc (row index 0):")
print(row_0)

print("\n")

# loc (label-based)
specific_val = df.loc[0, 'Name']
print("City of the person at index 0:")
print(specific_val)
```




```
Ages column:
0    25
1    30
2    35
Name: Age, dtype: int64


People with Age > 25:
      Name  Age    City
1      Bob   30   Paris
2  Charlie   35  London


First row using iloc (row index 0):
Name       Alice
Age           25
City    New York
Name: 0, dtype: object


City of the person at index 0:
Alice
```

Result: Thus, column selection, conditional row filtering, integer-based indexing using iloc, and label-based indexing using loc were performed successfully.

C. Data Cleaning (Handling Missing Values)

Aim: To identify and handle missing values in a Pandas DataFrame using isnull(), dropna(),

and fillna().

Algorithm:

1. Import the Pandas and NumPy libraries.
2. Create a dictionary containing Name, Age, and City with missing values represented by np.nan.
3. Convert the dictionary into a DataFrame.
4. Display the original DataFrame.
5. Count missing values in each column using df.isnull().sum().
6. Remove rows containing missing values using df.dropna().
7. Replace all missing values with zero using df.fillna(0).
8. Calculate the mean of the Age column and fill missing ages with the mean.
9. Replace missing values in the City column with "Unknown".
10. Display the results of each operation.

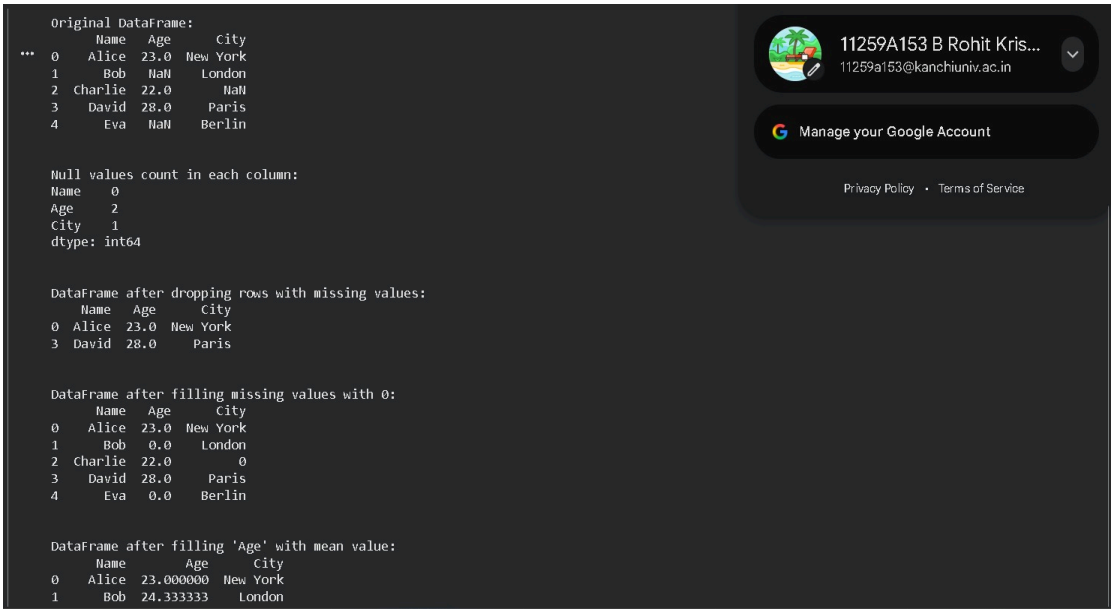
```python
import pandas as pd
import numpy as np

# Create sample dataset with missing values
data = {
    "Name": ["Alice", "Bob", "Charlie", "David", "Eva"],
    "Age": [23, np.nan, 22, 28, np.nan],
    "City": ["New York", "London", np.nan, "Paris", "Berlin"]
}
df = pd.DataFrame(data)
print("Original DataFrame:")
print(df)
print("\n")

# Check for null values
print("Null values count in each column:")
print(df.isnull().sum())
print("\n")

# Drop rows with missing values
df_clean = df.dropna()
print("DataFrame after dropping rows with missing values:")
print(df_clean)
print("\n")

# Fill missing values with 0
df_filled = df.fillna(0)
print("DataFrame after filling missing values with 0:")
print(df_filled)
print("\n")
```



Result: Thus, missing values were identified and handled successfully by removing incomplete rows, replacing missing values with zero, filling missing ages with the column mean, and replacing missing city values with "Unknown".